Лабораторна робота № 5-6

Нейронна мережа і регуляризація

Мета роботи

Реалізувати повнозв'язну нейронну мережу (MLP) з нуля, дослідити механізм зворотного поширення помилки, навчитися зберігати та відновлювати параметри нейронної мережі. Експериментально дослідити вплив різних методів регуляризації на розподіл ваг, розрідженість моделі, кількість неактивних нейронів та якість класифікації.

Основну увагу приділити:

- самостійній реалізації прямого та зворотного проходів нейронної мережі;
- розумінню того, як ваги та градієнти змінюються під час навчання;
- реалізації механізмів збереження та відновлення ваг;
- дослідженню впливу L1, L2, Dropout, weight clipping і gradient clipping;
- аналізу явища dead neurons для функції активації ReLU;
- пошуку залежності між розрідженістю ваг, кількістю неактивних нейронів та точністю моделі.

Очікувані результати навчання

Після виконання роботи студент повинен уміти:

1. реалізовувати повнозв'язні нейронні мережі за допомогою матричних операцій;
2. самостійно реалізовувати алгоритм backpropagation для обчислення градієнтів;
3. реалізовувати цикл навчання нейронної мережі та оптимізатор градієнтного спуску;
4. зберігати параметри навченої мережі та відновлювати її стан із файлу;
5. пояснювати математичну сутність L1 та L2 регуляризації;
6. розрізняти dead neurons і sparsity ваг;
7. аналізувати вплив різних методів регуляризації на параметри та поведінку нейронної мережі.

Програмне забезпечення

Python $\geq$ 3.10, NumPy, Matplotlib, scikit-learn. Для додаткових експериментів дозволяється використовувати PyTorch або JAX, однак реалізація прямого проходу, зворотного поширення помилки та обчислення градієнтів у першому блоці має бути виконана самостійно.

Заборонено використовувати вбудовані механізми автоматичного диференціювання та готові реалізації нейронних мереж для основної реалізації першого блоку.

1 Короткі теоретичні відомості

1.1 Повнозв'язна нейронна мережа

Повнозв'язна нейронна мережа складається з послідовності шарів. Для кожного шару обчислюється:

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}, \tag{1}$$

$$A^{(l)} = f(Z^{(l)}), \tag{2}$$

де $W^{(l)}$ — матриця ваг, $b^{(l)}$ — вектор зміщень, f — функція активації, а $A^{(l)}$ — вихід шару.

Для прихованих шарів використовується ReLU:

$$\mathrm{ReLU}(z) = \max(0, z). \tag{3}$$

Для вихідного шару багатокласової класифікації використовується Softmax:

$$\mathrm{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}. \tag{4}$$

Як функцію втрат використати categorical cross-entropy:

$$L_{\mathrm{CE}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,y_i}, \tag{5}$$

де p_{i,y_i} — передбачена моделлю ймовірність правильного класу для i-го об'єкта.

1.2 Зворотне поширення помилки

Backpropagation дозволяє обчислити похідні функції втрат за всіма параметрами мережі, використовуючи правило диференціювання складеної функції.

Для Softmax разом із cross-entropy градієнт за логітами має вигляд:

$$\frac{\partial L}{\partial Z^{(L)}} = \frac{P - Y}{N}, \tag{6}$$

де P — матриця передбачених імовірностей, а Y — матриця one-hot кодування правильних класів.

Для прихованих шарів:

$$\delta^{(l)} = \left(\delta^{(l+1)}(W^{(l+1)})^T\right) \odot f'(Z^{(l)}). \tag{7}$$

Для ReLU:

$$f'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \le 0. \end{cases} \tag{8}$$

Градієнти параметрів:

$$\frac{\partial L}{\partial W^{(l)}} = (A^{(l-1)})^T \delta^{(l)}, \tag{9}$$

$$\frac{\partial L}{\partial b^{(l)}} = \sum_{i=1}^{N} \delta_i^{(l)}. \tag{10}$$

Оновлення параметрів за допомогою SGD:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \tag{11}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}, \tag{12}$$

де η — швидкість навчання.

1.3 Ініціалізація ваг

Для початкової ініціалізації використати метод He:

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right), \tag{13}$$

де n_{in} — кількість входів шару.

Також можна використати Xavier:

$$W_{ij}^{(l)} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right). \tag{14}$$

Для мереж із ReLU рекомендовано використовувати He як основний метод ініціалізації.

1.4 Проблема мертвих нейронів

Нейрон із ReLU вважається мертвим, якщо його вихід дорівнює нулю для всіх об'єктів досліджуваної вибірки:

$$A_j^{(l)}(x_i) = 0 \quad \forall i \in \{1, \ldots, N\}. \tag{15}$$

Це означає, що передактивація нейрона не є додатною для жодного об'єкта:

$$Z_j^{(l)}(x_i) \leq 0 \quad \forall i. \tag{16}$$

Такий нейрон не передає сигнал далі та не отримує градієнта через ReLU. Він може перестати навчатися, особливо якщо параметри потрапили в несприятливу область.

Частка мертвих нейронів у шарі:

$$D_l = \frac{N_{\text{dead}}^{(l)}}{N_{\text{neurons}}^{(l)}} \cdot 100\%. \tag{17}$$

Важливо: нейрон, який не активувався на тренувальній вибірці, не обов'язково є мертвим для всього можливого простору даних.

1.5 L1 та L2 регуляризація

L1 регуляризація додає до функції втрат суму абсолютних значень ваг:

$$L = L_{\text{CE}} + \lambda_1 \sum_l |W^{(l)}|_1. \tag{18}$$

Вона сприяє формуванню розріджених матриць ваг, оскільки може приводити частину параметрів до точного або майже нульового значення.

L2 регуляризація додає суму квадратів ваг:

$$L = L_{\text{CE}} + \frac{\lambda_2}{2} \sum_l |W^{(l)}|_2^2. \tag{19}$$

Вона штрафує великі значення ваг і зазвичай сприяє їх зменшенню без такого вираженого формування точних нулів, як у випадку L1.

1.6 Dropout

Dropout випадково вимикає частину нейронів під час навчання. Для кожного елемента активації генерується маска:

$$M_i \sim \text{Bernoulli}(1-p), \tag{20}$$

де p — ймовірність вимкнення нейрона.

За inverted dropout активації обчислюються як:

$$\widetilde{A} = \frac{A \odot M}{1-p}. \tag{21}$$

Під час перевірки та тестування Dropout вимикається.

Dropout не слід ототожнювати з dead neurons: нейрони, вимкнені випадковою маскою під час окремого кроку навчання, не є мертвими.

1.7 Weight clipping та gradient clipping

Weight clipping обмежує значення параметрів заданим діапазоном:

$$W_{ij} \leftarrow \min(1, \max(-1, W_{ij})). \tag{22}$$

Gradient clipping обмежує норму градієнта:

$$g \leftarrow g \cdot \min\left(1, \frac{c}{|g|_2}\right), \tag{23}$$

де c — максимально допустима норма градієнта.

На відміну від регуляризації L1 та L2, clipping безпосередньо не додає штраф до функції втрат. Він змінює параметри або градієнти під час навчання.

1.8 Розрідженість ваг

Для оцінювання розрідженості використати частку ваг, модуль яких менший за заданий поріг:

$$S = \frac{\#\{|W_{ij}| < 0.01\}}{\#W_{ij}} \cdot 100\%. \tag{24}$$

Висока sparsity не означає автоматично високу якість моделі. Так само велика кількість мертвих нейронів не обов'язково є прямим наслідком великої кількості малих ваг.

2 Блок 1. Реалізація нейронної мережі з нуля

2.1 Архітектура мережі

Реалізувати багатошарову повнозв'язну нейронну мережу для задачі багатокласової класифікації.

Використати:

- вхідний шар відповідно до розмірності датасету;
- 3–4 приховані шари;

- ReLU для прихованих шарів;
- Softmax для вихідного шару;
- categorical cross-entropy як функцію втрат;
- SGD як алгоритм оптимізації;
- He initialization для початкових ваг.

Як датасет використати FashionMNIST. Зображення необхідно перетворити на вектори розмірності $28 \cdot 28 = 784$ та нормалізувати значення пікселів до діапазону $[0, 1]$.

Для прискорення експериментів дозволяється використовувати частину тренувального набору, але всі експерименти повинні використовувати однакові підмножини даних.

2.2 Реалізація прямого проходу

Реалізувати клас нейронної мережі, який зберігає:

- матриці ваг усіх шарів;
- вектори зміщень;
- проміжні значення $Z^{(l)}$ та $A^{(l)}$;
- параметри архітектури мережі.

Реалізувати метод прямого проходу, який приймає батч об'єктів та повертає ймовірності належності до класів.

```
def forward(self, X):
    self.activations = [X]
    self.z_values = []

    A = X

    for W, b in zip(self.weights[:-1], self.biases[:-1]):
        Z = A @ W + b
        A = np.maximum(0, Z)

        self.z_values.append(Z)
        self.activations.append(A)

    Z = A @ self.weights[-1] + self.biases[-1]

    Z = Z - np.max(Z, axis=1, keepdims=True)
    exp_Z = np.exp(Z)
    P = exp_Z / np.sum(exp_Z, axis=1, keepdims=True)

    self.z_values.append(Z)
    self.activations.append(P)

    return P
```

2.3 Реалізація backpropagation

Самостійно реалізувати обчислення градієнтів для всіх матриць ваг і векторів зміщень.
Не використовувати:

- `torch.autograd`;
- `torch.nn.Module`;
- `jax.grad`;
- готові реалізації оптимізаторів;
- бібліотеки автоматичного диференціювання.

Дозволяється використовувати NumPy для матричних операцій.
Реалізація повинна:

1. обчислювати градієнт вихідного шару;
2. послідовно поширювати градієнт у зворотному напрямку;
3. враховувати похідну ReLU;
4. обчислювати градієнти за вагами та зміщеннями;
5. повертати градієнти для подальшого оновлення параметрів.

```python
def backward(self, y_true):
    n = y_true.shape[0]

    grads_W = [None] * len(self.weights)
    grads_b = [None] * len(self.biases)

    # Softmax + Cross Entropy
    delta = self.activations[-1].copy()
    delta[np.arange(n), y_true] -= 1
    delta /= n

    for l in reversed(range(len(self.weights))):
        A_prev = self.activations[l]

        grads_W[l] = A_prev.T @ delta
        grads_b[l] = np.sum(delta, axis=0)

        if l > 0:
            delta = delta @ self.weights[l].T
            delta *= (self.z_values[l - 1] > 0)

    return grads_W, grads_b
```

2.4 Цикл навчання

Реалізувати повний цикл навчання:

1. перемішування тренувальних даних;
2. поділ на мінібатчі;

3. прямий прохід;

4. обчислення функції втрат;

5. зворотне поширення помилки;

6. оновлення ваг та зміщень;

7. оцінювання якості на валідаційній вибірці.

Для оновлення параметрів використати:

$$W^{(l)} \leftarrow W^{(l)} - \eta \nabla_{W^{(l)}} L. \tag{25}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \nabla_{b^{(l)}} L. \tag{26}$$

Побудувати графіки:

- функції втрат на тренувальній і валідаційній вибірках;
- accuracy на тренувальній і валідаційній вибірках.

2.5 Перевірка правильності градієнтів

Перевірити реалізацію backpropagation методом чисельного диференціювання.

Для параметра θ чисельна оцінка градієнта:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \varepsilon) - L(\theta - \varepsilon)}{2\varepsilon}. \tag{27}$$

Порівняти чисельні та аналітичні градієнти для декількох випадково вибраних параметрів.

Використати відносну похибку:

$$E = \frac{|g_{\text{analytic}} - g_{\text{numeric}}|}{\max(10^{-8}, |g_{\text{analytic}}| + |g_{\text{numeric}}|)}. \tag{28}$$

2.6 Збереження та відновлення ваг

Реалізувати механізм збереження навченої нейронної мережі у файл та її подальшого відновлення.

Зберігати:

- усі матриці ваг;
- усі вектори зміщень;
- архітектуру мережі;
- параметри ініціалізації;
- необхідні параметри конфігурації моделі.

Для збереження дозволяється використовувати `numpy.savez` або інший формат серіалізації.

Приклад збереження:

```python
def save_weights(self, path):
    data = {}

    for i, W in enumerate(self.weights):
        data[f'W_{i}'] = W

    for i, b in enumerate(self.biases):
        data[f'b_{i}'] = b

    np.savez(path, **data)
```

Реалізувати метод завантаження ваг у новий екземпляр нейронної мережі.

```python
def load_weights(self, path):
    data = np.load(path)

    self.weights = [
        data[f'W_{i}']
        for i in range(self.num_layers)
    ]

    self.biases = [
        data[f'b_{i}']
        for i in range(self.num_layers)
    ]
```

Перевірити коректність відновлення:

1. навчити нейронну мережу;
2. зберегти її ваги у файл;
3. створити новий екземпляр мережі з такою самою архітектурою;
4. завантажити збережені ваги;
5. передати однакові вхідні дані обом мережам;
6. порівняти вихідні ймовірності та передбачені класи.

Для коректного відновлення виходи двох мереж повинні збігатися з урахуванням похибки обчислень:

$$\max |P_{\text{original}} - P_{\text{restored}}| < 10^{-7}. \tag{29}$$

3 Блок 2. Дослідження впливу регуляризації

3.1 Базовий експеримент

Використати реалізовану в першому блоці нейронну мережу з 3–4 прихованими шарами та функцією активації ReLU.

Навчити модель на FashionMNIST без використання будь-яких методів регуляризації.

Зафіксувати:

- фінальну accuracy на валідаційній вибірці;
- значення функції втрат;

- кількість мертвих нейронів у кожному прихованому шарі;
- розподіл ваг кожного шару;
- частку майже нульових ваг.

Цей експеримент буде базовою точкою для порівняння з усіма наступними методами регуляризації.

3.2 Діагностика мертвих нейронів

Після навчання перевірити активації кожного прихованого шару на всій тренувальній вибірці.

Нейрон вважається мертвим, якщо його передактивація не є додатною для жодного об'єкта тренувальної вибірки.

Для кожного прихованого шару обчислити:

$$N_{\text{dead}}^{(l)} = \sum_j \mathbb{I}\left[\max_i Z_{ij}^{(l)} \leq 0\right]. \tag{30}$$

Частку мертвих нейронів визначити як:

$$D_l = \frac{N_{\text{dead}}^{(l)}}{N_{\text{neurons}}^{(l)}} \cdot 100\%. \tag{31}$$

Результати подати у вигляді таблиці для всіх прихованих шарів.

Важливо. Під час діагностики Dropout необхідно вимкнути. Оцінювання проводиться в режимі inference, щоб випадково вимкнені нейрони не були помилково класифіковані як мертві.

3.3 Дослідження L2 регуляризації

Повторити навчання базової мережі з L2 регуляризацією.

$$L = L_{\text{CE}} + \frac{\lambda_2}{2} \sum_l |W^{(l)}|_2^2. \tag{32}$$

Для оновлення ваг використати:

$$W^{(l)} \leftarrow W^{(l)} - \eta \left(\nabla_{W^{(l)}} L_{\text{CE}} + \lambda_2 W^{(l)}\right). \tag{33}$$

Дослідити декілька значень коефіцієнта регуляризації λ_2.

Порівняти:

- розподіл ваг із базовою моделлю;
- кількість мертвих нейронів;
- частку майже нульових ваг;
- фінальну валідаційну точність.

3.4 Дослідження L1 регуляризації

Додати до функції втрат L1 штраф:

$$L = L_{\text{CE}} + \lambda_1 \sum_l |W^{(l)}|_1. \tag{34}$$

Градієнт регуляризаційної складової:

$$\frac{\partial L_{\text{reg}}}{\partial W^{(l)}} = \lambda_1 \operatorname{sign}(W^{(l)}). \tag{35}$$

Під час реалізації врахувати, що в точці нуля похідна модуля не визначена. Для обчислень використати субградієнт, рівний нулю в цій точці.

Дослідити вплив різних значень λ_1 на:

- розрідженість матриць ваг;
- розподіл ваг;
- кількість мертвих нейронів;
- валідаційну точність.

3.5 Дослідження Dropout

Реалізувати Dropout для прихованих шарів нейронної мережі.

Виконати окремі експерименти для таких значень імовірності вимкнення нейронів:

$$p \in \{0.1,\ 0.3,\ 0.5\}. \tag{36}$$

Для кожного значення:

1. навчити мережу з Dropout;
2. оцінити кількість мертвих нейронів;
3. обчислити sparsity ваг;
4. побудувати гістограми розподілу ваг;
5. визначити фінальну валідаційну accuracy.

Порівняти результати та пояснити, чому збільшення ймовірності Dropout не обов'язково покращує якість моделі.

3.6 Дослідження Weight clipping

Реалізувати обмеження ваг після кожного кроку оновлення:

$$W^{(l)} \leftarrow \text{clip}(W^{(l)}, -1, 1). \tag{37}$$

Повторити навчання мережі з цим обмеженням.

Дослідити:

- зміну розподілу ваг;
- кількість ваг, що досягли меж -1 та 1;
- частку майже нульових ваг;

- кількість мертвих нейронів;
- фінальну точність.

Пояснити, як жорстке обмеження параметрів впливає на можливість моделі знаходити оптимальні значення ваг.

3.7 Дослідження Gradient clipping

Реалізувати обрізання градієнтів за глобальною L2-нормою:

$$g_{\text{clipped}} = g \cdot \min\left(1, \frac{c}{|g|_2}\right). \tag{38}$$

Виконати експерименти з декількома значеннями порога c.
Порівняти:

- стабільність функції втрат;
- розподіл ваг;
- частку майже нульових ваг;
- кількість мертвих нейронів;
- фінальну валідаційну точність.

Звернути увагу, що gradient clipping обмежує градієнти, а не самі ваги, тому його вплив на розподіл параметрів відрізняється від weight clipping.

3.8 Комбінована регуляризація

Дослідити спільне використання декількох методів регуляризації.
Обов'язково реалізувати комбінацію:

$$L = L_{\text{CE}} + \frac{\lambda_2}{2} \sum_l |W^{(l)}|_2^2 \tag{39}$$

разом із Dropout із параметром:

$$p = 0.2. \tag{40}$$

Порівняти комбінований підхід із відповідними окремими експериментами L2 та Dropout.
Визначити:

- чи покращилася валідаційна точність;
- чи зменшилася кількість мертвих нейронів;
- як змінилася sparsity ваг;
- чи спостерігається синергія між методами або, навпаки, надмірне обмеження моделі.

4 Аналіз та порівняння результатів

4.1 Порівняльна таблиця

Для кожного експерименту сформувати таблицю з такими показниками:

Метод	Dead neurons, %	Sparsity, %	Val. accuracy	Val. loss
Без регуляризації				
L1				
L2				
Dropout 0.1				
Dropout 0.3				
Dropout 0.5				
Weight clipping				
Gradient clipping				
L2 + Dropout 0.2				

Для методів, що потребують налаштування коефіцієнтів, вказати використані значення λ_1, λ_2 та порога clipping.

4.2 Розподіл ваг

Для кожного методу побудувати гістограми розподілу ваг усіх прихованих шарів.

На графіках порівняти:

- початковий розподіл ваг після He initialization;
- розподіл ваг після навчання без регуляризації;
- розподіл ваг після застосування кожного методу.

Окремо проаналізувати, як змінюється кількість ваг, близьких до нуля, та чи виникають значні піки розподілу біля меж -1 і 1 у випадку weight clipping.

4.3 Залежність мертвих нейронів від точності

Побудувати діаграму розсіювання:

- вісь x — відсоток мертвих нейронів;
- вісь y — валідаційна accuracy;
- кожна точка — окремий експеримент.

На графіку підписати назви методів регуляризації.

За результатами графіка дослідити:

1. чи існує залежність між кількістю мертвих нейронів і якістю класифікації;
2. чи призводить велика кількість мертвих нейронів до зниження точності;
3. чи можуть моделі з однаковою кількістю мертвих нейронів мати різну точність;
4. чи існує оптимальний баланс між sparsity ваг і кількістю мертвих нейронів.

4.4 Дослідження залежності регуляризації від якості

На основі всіх експериментів відповісти на дослідницьке питання:

> Чи завжди збільшення сили регуляризації покращує узагальнювальну здатність нейронної мережі?

Порівняти результати слабкої та сильної регуляризації. Визначити, за яких умов регуляризація знижує перенавчання, а за яких призводить до недонавчання.

Окремо пояснити різницю між такими поняттями:

- мала величина ваг;
- нульова або майже нульова вага;
- неактивний нейрон;
- мертвий нейрон;
- надмірно регуляризована модель.

5 Вимоги до звіту

Звіт оформити у форматі Jupyter Notebook.

Звіт повинен містити:

Блок 1. Реалізація нейронної мережі

1. архітектуру нейронної мережі;
2. реалізацію прямого проходу;
3. реалізацію backpropagation;
4. перевірку градієнтів чисельним диференціюванням;
5. графіки навчання;
6. реалізацію збереження ваг;
7. перевірку відновлення мережі зі збережених ваг.

Блок 2. Дослідження регуляризації

1. результати базового експерименту;
2. порівняльну таблицю всіх методів регуляризації;
3. гістограми розподілу ваг;
4. таблиці dead neurons для кожного прихованого шару;
5. порівняння sparsity;
6. графік залежності dead neurons від validation accuracy;
7. результати комбінованої регуляризації;
8. аналіз отриманих результатів.

Усі графіки та таблиці повинні супроводжуватися коротким поясненням отриманих результатів.